

13_unsupervised_learning_filled

May 25, 2026

1 CSE 25 - Introduction to Artificial Intelligence

1.1 Week 9 Tuesday: Unsupervised Learning

Learning Objectives:

- Distinguish unsupervised learning from supervised learning and reinforcement learning.
- Describe the K-means algorithm: the two-step assign/update loop and the role of centroids.
- Implement the K-means algorithm by hand: `squared_euclidean_distance`, `assign_clusters`, `update_centroids`.
- Identify underfitting and overfitting in K-means in terms of the choice of K.
- Use the elbow method to choose a reasonable value of K.
- Use distance-to-centroid as an outlier score to detect unusual data points.

Instructions

Use your copy of this notebook on Datahub and complete it during class. Work through the cells below **in order**. You may discuss with your neighbors, but make sure you understand each step yourself.

SUBMISSION: When finished, download the notebook to have a local copy for your records. Choose two cells where you wrote code or answers and take screenshots of them to upload to Gradescope under **In-Class - Week 9 Tuesday** to receive credit.

Today's focus:

What can a machine learn from data when it is given **no labels** and **no rewards** — just the raw data itself?

1.1.1 Three Ways to Learn

Over the past several weeks, we have studied three different learning paradigms. Here is how they compare:

Paradigm	Input	Feedback	Goal
Supervised learning	Features + labels	Correct answer for each example	Predict labels for new examples
Reinforcement learning	State of the environment	Reward signal after actions	Learn a policy that maximizes reward
Unsupervised learning	Features only <i>no labels, no rewards</i>	None	Discover hidden structure in the data

In **unsupervised learning**, no one tells the algorithm what the “right” answer is. The goal is to find patterns, groups, or unusual points that are hidden in the data.

Common tasks: - **Clustering**: group similar examples together (e.g., group customers by purchase behavior) - **Outlier / anomaly detection**: find unusual examples that don’t fit any group (e.g., fraudulent transactions, faulty sensors) - **Dimensionality reduction**: find a compact representation of the data

Today we will focus on **clustering** and **outlier detection** using the **K-means** algorithm, coded from scratch.

1.1.2 Handwritten Digits

You used the **digits dataset** in PA1, where you built a perceptron to classify digits from their pixel images. This time, we will use the **same dataset** but see what we can discover when we have no labels.

The question we will answer: Can K-means discover which images are 0s, 1s, and 2s without ever being told?

Each image is an 8×8 grid of pixels. We will work with a **small subset** of 30 images (10 each of digits 0, 1, and 2) and summarize each image with **two interpretable features**:

- **top_intensity**: the mean pixel brightness of the **top 4 rows** of the image
- **bottom_intensity**: the mean pixel brightness of the **bottom 4 rows** of the image

Intuitively: - A **0** is a closed oval so it’s roughly symmetric top/bottom - A **1** is a vertical stroke, which means it’s sparse overall, slightly concentrated in the middle - A **2** has a horizontal stroke at the bottom and has more weight in the lower half

```
[1]: import math
import random
import matplotlib.pyplot as plt
from sklearn import datasets

# Load the digits dataset and convert to plain Python lists.
digits = datasets.load_digits()

# Each image is a list of 8 rows, each row is a list of 8 pixel values
all_images = digits.images.tolist()    # list of 1797 images;
all_labels = digits.target.tolist()    # list of 1797 integers (0-9)

print(f"Number of images: {len(all_images)}")
print(f"First label: {all_labels[0]}")
print(f"First image (8 rows of 8 pixels each):")
for row in all_images[0]:
    print([round(v) for v in row])
```

Number of images: 1797

First label: 0

First image (8 rows of 8 pixels each):

```
[0, 0, 5, 13, 9, 1, 0, 0]
[0, 0, 13, 15, 10, 15, 5, 0]
[0, 3, 15, 2, 0, 11, 8, 0]
[0, 4, 12, 0, 0, 8, 8, 0]
[0, 5, 8, 0, 0, 9, 8, 0]
[0, 4, 11, 0, 1, 12, 7, 0]
[0, 2, 14, 5, 10, 12, 0, 0]
[0, 0, 6, 13, 10, 0, 0, 0]
```

Q: How do the pixel values for the first image relate to its label?

YOUR ANSWER HERE

```
[2]: def get_indices_for_digit(labels, digit, n=10):
    '''Return the first n indices where labels[i] == digit.'''
    result = []
    for i, label in enumerate(labels):
        if label == digit and len(result) < n:
            result.append(i)
    return result

# Collect 10 examples each of digits 0, 1, and 2
indices_0 = get_indices_for_digit(all_labels, 0, 10)
indices_1 = get_indices_for_digit(all_labels, 1, 10)
indices_2 = get_indices_for_digit(all_labels, 2, 10)

subset_indices = indices_0 + indices_1 + indices_2 # 30 indices total
X_images = [all_images[i] for i in subset_indices] # list of 30 images
y_true = [all_labels[i] for i in subset_indices] # list of 30 true labels

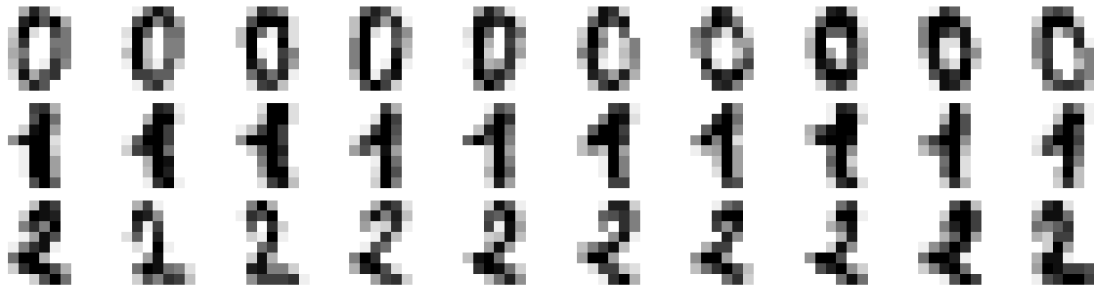
print(f"Subset size: {len(X_images)} images")
print(f"True labels: {y_true}")
```

Subset size: 30 images

True labels: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2]

```
[3]: # Display the 30 images in the subset (labels hidden)
fig, axes = plt.subplots(3, 10, figsize=(14, 4))
for i, ax in enumerate(axes.flat):
    ax.imshow(X_images[i], cmap='gray_r') # imshow works with a list-of-lists
    ax.axis('off')
plt.suptitle("30 handwritten digit images (labels hidden)", fontsize=13)
plt.tight_layout()
plt.show()
```

30 handwritten digit images (labels hidden)



Q: Just by looking at the images, can you identify which digits are 0, 1, and 2? Which examples are hardest to label, and why?

YOUR ANSWER HERE

```
[4]: def image_top_intensity(image):
    '''Mean pixel value in the top 4 rows (rows 0-3) of an 8x8 image.'''
    top_pixels = []
    for row in range(4):
        for col in range(8):
            top_pixels.append(image[row][col])
    return sum(top_pixels) / len(top_pixels)

def image_bottom_intensity(image):
    '''Mean pixel value in the bottom 4 rows (rows 4-7) of an 8x8 image.'''
    bottom_pixels = []
    for row in range(4, 8):
        for col in range(8):
            bottom_pixels.append(image[row][col])
    return sum(bottom_pixels) / len(bottom_pixels)

# Build the feature list: X is a list of [top_intensity, bottom_intensity] pairs
X = [[image_top_intensity(img), image_bottom_intensity(img)] for img in
     ↪X_images]

print(f"Number of data points: {len(X)}")
print("First 5 feature vectors [top_intensity, bottom_intensity]:")
for i in range(5):
    print(f"  Point {i}: {[round(v, 2) for v in X[i]]} (true label:
    ↪{y_true[i]})")
```

Number of data points: 30

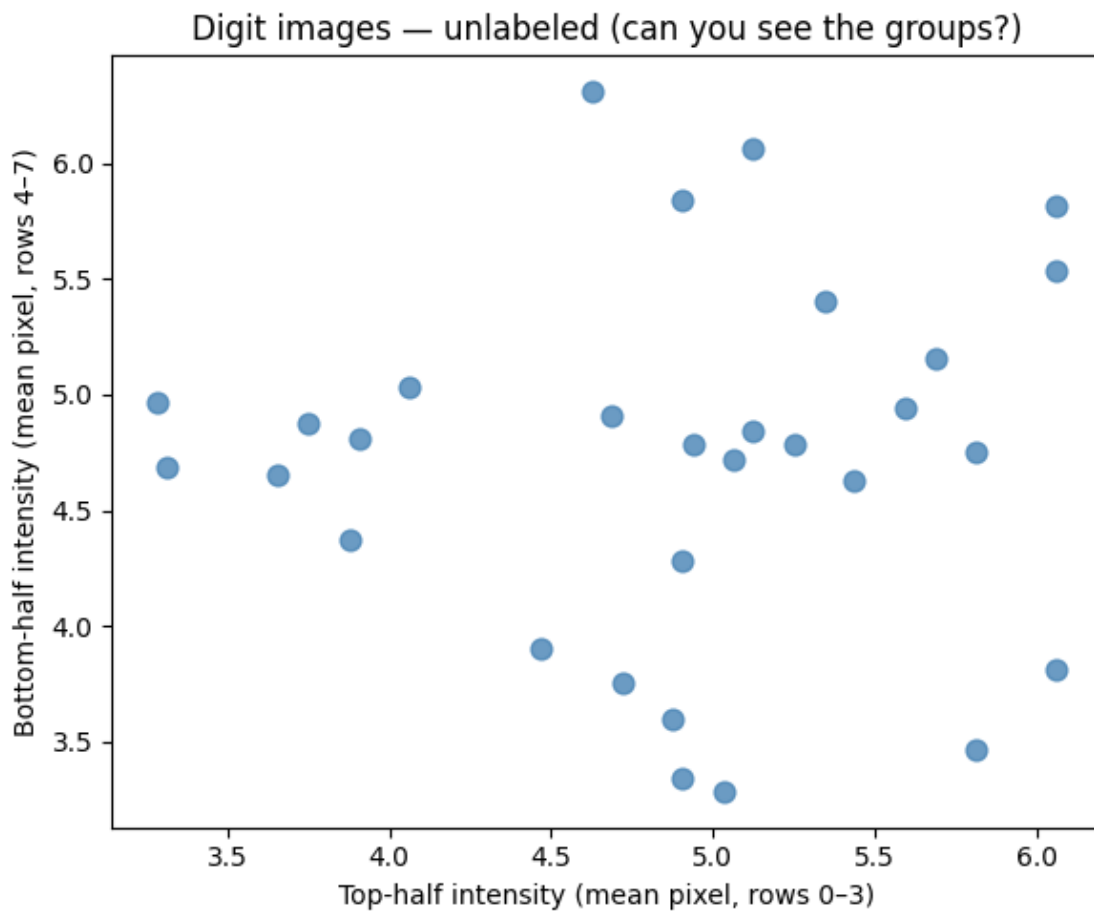
First 5 feature vectors [top_intensity, bottom_intensity]:

```
Point 0: [4.91, 4.28] (true label: 0)
Point 1: [5.44, 4.62] (true label: 0)
Point 2: [5.59, 4.94] (true label: 0)
```

Point 3: [5.34, 5.41] (true label: 0)
Point 4: [5.69, 5.16] (true label: 0)

```
[5]: # Extract x and y coordinates as plain lists for plotting
x_coords = [point[0] for point in X]
y_coords = [point[1] for point in X]

plt.figure(figsize=(6, 5))
plt.scatter(x_coords, y_coords, alpha=0.8, s=60, color='steelblue')
plt.xlabel("Top-half intensity (mean pixel, rows 0-3)")
plt.ylabel("Bottom-half intensity (mean pixel, rows 4-7)")
plt.title("Digit images - unlabeled (can you see the groups?)")
plt.tight_layout()
plt.show()
```



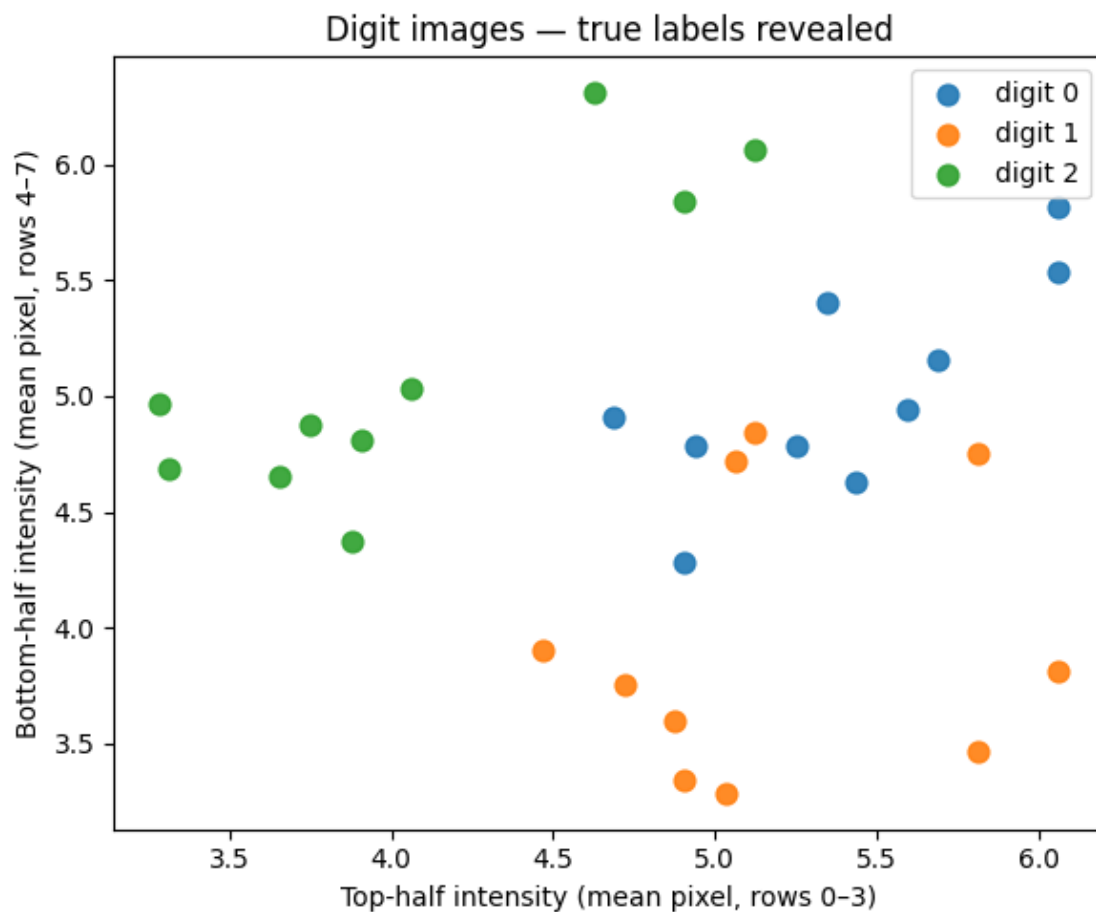
Q: Do you see any natural groups in the scatter plot above? How many groups, and how would you describe them?

YOUR ANSWER HERE

```
[6]: # Now reveal the true labels
colors = ['tab:blue', 'tab:orange', 'tab:green']
label_names = ['digit 0', 'digit 1', 'digit 2']

plt.figure(figsize=(6, 5))
for digit in range(3):
    xs = [X[i][0] for i in range(len(X)) if y_true[i] == digit]
    ys = [X[i][1] for i in range(len(X)) if y_true[i] == digit]
    plt.scatter(xs, ys, color=colors[digit], label=label_names[digit], alpha=0.
        ↪9, s=60)

plt.xlabel("Top-half intensity (mean pixel, rows 0-3)")
plt.ylabel("Bottom-half intensity (mean pixel, rows 4-7)")
plt.title("Digit images - true labels revealed")
plt.legend()
plt.tight_layout()
plt.show()
```



Q: Does the position of each digit in the feature space match your intuition about top vs. bottom intensity? Explain.

YOUR ANSWER HERE

1.1.3 The K-Means Algorithm

K-means is an algorithm for finding K **clusters** in unlabeled data. Each cluster is represented by a **centroid**, the mean of all the points in that cluster.

Starting from K randomly chosen centroids, the algorithm alternates between two steps until the centroids stop moving:

Step 1: Assign For each data point $x^{(j)}$, find the nearest centroid and assign that point to its cluster:

$$\text{assignment}(j) = \arg \min_k \|x^{(j)} - c_k\|$$

Step 2: Update For each cluster k , move the centroid to the mean of all assigned points:

$$c_k \leftarrow \frac{1}{|C_k|} \sum_{i: \text{assignment}(j)=k} x^{(j)}$$

We measure the goodness of a clustering based on how far away points in a cluster are from its centroid. The **total inertia** of a clustering is the sum of squared distances from each point to its nearest centroid:

$$J = \sum_{j=1}^n \|x^{(j)} - c_{\text{assignment}(j)}\|^2$$

A lower J means the clusters are tighter (points are closer to their centroid). The K -means algorithm minimizes total inertia.

Tracing K-Means on a Tiny Example Before coding, let's trace one iteration by hand. Consider these 6 points in 2D with $K = 2$:

Point	x_1	x_2
A	1	1
B	2	1
C	1.5	2
D	7	8
E	8	7
F	7.5	9

Suppose we initialize with centroids $c_1 = (1, 1)$ (point A) and $c_2 = (7, 8)$ (point D).

Q: After **Step 1: Assign**, which cluster does each of the 6 points belong to? (Hint: just look at whether each point is closer to (1,1) or to (7,8).)

YOUR ANSWER HERE

Q: After **Step 2: Update**, what are the new positions of centroids c_1 and c_2 ?

YOUR ANSWER HERE

Implementing K-Means We will now implement K-means from scratch using three functions.

Data representations we'll use: - A **data point** is a list: [top_intensity, bottom_intensity]
- The **dataset** X is a list of data points: [[x0, y0], [x1, y1], ...] - **centroids** is a list of K data points: [[cx0, cy0], [cx1, cy1], ...] - **assignments** is a list of integers: [0, 2, 1, 0, ...] (one cluster index per data point)

```
[7]: def squared_euclidean_distance(x, y):  
    '''  
    Returns the square of the Euclidean distance between two points x and y,  
    where x and y are lists of the same length.  
  
    Recall: distance = sqrt( sum of squared differences )  
    '''  
    # YOUR CODE HERE  
    # Hint: use math.sqrt and a loop or list comprehension over zip(x, y)  
    # REMOVE  
    total = 0.0  
    for xi, yi in zip(x, y):  
        total += (xi - yi) ** 2  
    return total  
  
# Test euclidean_distance function  
assert squared_euclidean_distance([0, 0], [3, 4]) == 25.0 , "should be 25.0"  
print(f"Squared Euclidean distance between [0,0] and [3,4] is_  
↪{squared_euclidean_distance([0, 0], [3, 4])}")  
assert squared_euclidean_distance([1,1], [1,1]) == 0 , "should be 0.0"  
print(f"Squared Euclidean distance between [1,1] and [1,1] is_  
↪{squared_euclidean_distance([1, 1], [1, 1])}")
```

Squared Euclidean distance between [0,0] and [3,4] is 25.0

Squared Euclidean distance between [1,1] and [1,1] is 0.0

```
[8]: def assign_clusters(X, centroids):  
    '''  
    Assign each point in X to the nearest centroids by computing index of_  
    ↪centroid with the smallest distance given  
    X, a list of n data points, each a list of d features and  
    centroids, a list of K centroids, each a list of d features  
  
    Returns list of n integers, where assignments[i] is the index (0 to K-1) of_  
    ↪the nearest centroid to X[i]  
    '''  
    assignments = []  
  
    for point in X:
```



```

# UNCOMMENT AND FILL IN:
# min_dist = float('inf')
# best_k = 0
# for k, centroid in enumerate(centroids):
#     dist = ...
#     if dist < min_dist:
#         ...
# assignments.append(best_k)

# REMOVE
min_dist = float('inf')
best_k = 0
for k, centroid in enumerate(centroids):
    dist = squared_euclidean_distance(point, centroid)
    if dist < min_dist:
        min_dist = dist
        best_k = k
assignments.append(best_k)

return assignments

# Test assign_clusters function
test_X = [[0.0, 0.0], [10.0, 10.0], [0.5, 0.5]]
test_centroids = [[0.0, 0.0], [10.0, 10.0]]
assert assign_clusters(test_X, test_centroids) == [0,1,0] , "should be [0,1,0]"

print(f"First two points are the centroids of the 0th and 1th clusters, third_
↪point is closer to first point then second, so {assign_clusters(test_X,
↪test_centroids)}")

```

First two points are the centroids of the 0th and 1th clusters, third point is closer to first point then second, so [0, 1, 0]

```

[9]: def update_centroids(X, assignments, K):
    """
    Compute the new centroid for each of K clusters as the mean of its assigned_
    ↪points, given X list of n data points, each a list of d features, and
    ↪assignments list of n integers (cluster index for each point)

    Returns: list of new centroids (each a list of d features) for the K_
    ↪clusters
    """
    d = len(X[0]) # number of features per point
    new_centroids = []

    for k in range(K):
        # UNCOMMENT AND FILL IN:

```

```

# Collect all points where assignments[i] == k
# points_in_k = [X[i] for i in range(len(X)) if assignments[i] == k]
# if len(points_in_k) > 0:
#     centroid = [ ... ] # list of d feature means
# else:
#     centroid = [0.0] * d
# new_centroids.append(centroid)

# REMOVE
points_in_k = [X[i] for i in range(len(X)) if assignments[i] == k]
if len(points_in_k) > 0:
    centroid = [sum(pt[j] for pt in points_in_k) / len(points_in_k)
                for j in range(d)]
else:
    centroid = [0.0] * d
new_centroids.append(centroid)

return new_centroids

# Test update_centroids function
test_X2 = [[0.0, 0.0], [2.0, 2.0], [10.0, 10.0], [12.0, 12.0]]
test_asgn = [0, 0, 1, 1]
result = update_centroids(test_X2, test_asgn, K=2)
assert result == [[1.0, 1.0], [11.0, 11.0]], "should be [[1, 1], [11, 11]]"

print(f"First two points are in the same cluster so update that cluster_
↪centroid to the means of their features [(0+2)/2,(0+2)/2].")
print(f"Last two points are in the same cluster so update that cluster centroid_
↪to the means of their features [(10+12)/2,(10+12)/2].")
print(f"Updated centroids are {result}")

```

First two points are in the same cluster so update that cluster centroid to the means of their features $[(0+2)/2, (0+2)/2]$.

Last two points are in the same cluster so update that cluster centroid to the means of their features $[(10+12)/2, (10+12)/2]$.

Updated centroids are $[[1.0, 1.0], [11.0, 11.0]]$

```

[10]: def compute_inertia(X, assignments, centroids):
    """
    The total inertia of data points X with assignments to clusters is the sum_
    ↪of squared distances from each point to its assigned centroid.
    """
    total = 0.0
    for i in range(len(X)):
        k = assignments[i]
        total += squared_euclidean_distance(X[i], centroids[k])
    return total

```

```

def centroids_converged(old_centroids, new_centroids, tol=1e-9):
    '''Returns True if every centroid moved by less than tol.'''
    for old, new in zip(old_centroids, new_centroids):
        if squared_euclidean_distance(old, new) > tol:
            return False
    return True

def run_kmeans(X, K, max_iter=50, seed=0):
    '''
    Run K-means clustering and returns:
    assignments      : list of n cluster labels (integers)
    centroids        : list of K final centroid positions
    inertia_history  : list of inertia values, one per iteration
    '''
    random.seed(seed)

    # Initialize: choose K random data points as starting centroids
    init_indices = random.sample(range(len(X)), K)
    centroids = [X[i][:] for i in init_indices] # copy the selected points

    inertia_history = []

    for iteration in range(max_iter):
        # Step 1: Assign each point to its nearest centroid
        assignments = assign_clusters(X, centroids)

        # Compute and record the current inertia
        inertia = compute_inertia(X, assignments, centroids)
        inertia_history.append(inertia)

        # Step 2: Update centroids to the mean of each cluster
        new_centroids = update_centroids(X, assignments, K)

        # Early stop if centroids didn't move (convergence)
        if centroids_converged(centroids, new_centroids):
            print(f"K-means converged at iteration {iteration + 1}")
            break

        centroids = new_centroids

    return assignments, centroids, inertia_history

# Run K-means with K=3 on our 30-point digit subset

```

```

assignments, centroids, inertia_history = run_kmeans(X, K=3, seed=42)

print("\nCluster assignments:", assignments)
print("\nFinal centroids:")
for k in range(3):
    print(f"    Centroid {k}: top={centroids[k][0]:.2f}, bottom={centroids[k][1]:.2f}")

```

K-means converged at iteration 5

Cluster assignments: [2, 1, 1, 1, 1, 1, 2, 1, 1, 1, 1, 1, 1, 2, 2, 1, 2, 1, 2, 2, 0, 2, 2, 2, 2, 2, 2, 2, 0, 0]

Final centroids:

Centroid 0: top=4.89, bottom=6.07

Centroid 1: top=5.56, bottom=4.82

Centroid 2: top=4.25, bottom=4.32

```

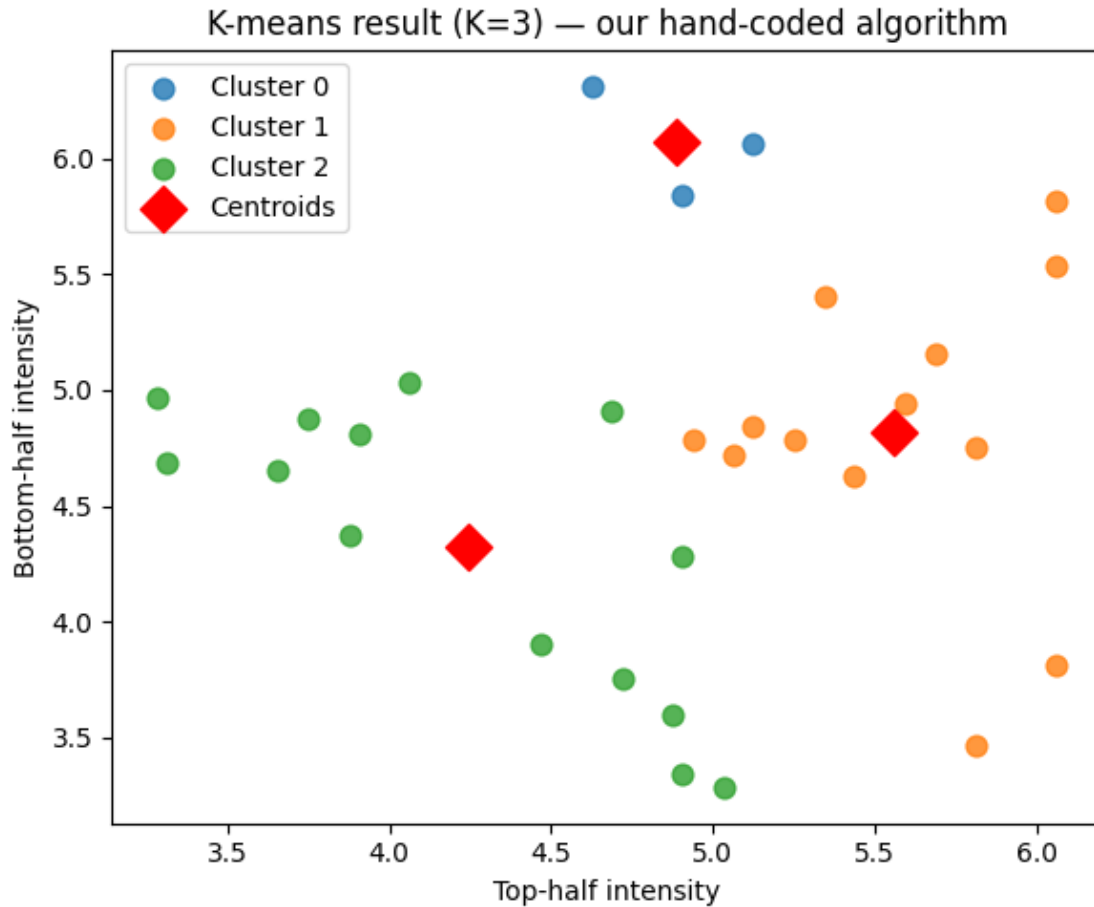
[11]: # Visualize the K-means result
cluster_colors = ['tab:blue', 'tab:orange', 'tab:green']

plt.figure(figsize=(6, 5))
for k in range(3):
    xs = [X[i][0] for i in range(len(X)) if assignments[i] == k]
    ys = [X[i][1] for i in range(len(X)) if assignments[i] == k]
    plt.scatter(xs, ys, color=cluster_colors[k], label=f'Cluster {k}', alpha=0.8, s=60)

# Mark centroids as diamonds
cx = [c[0] for c in centroids]
cy = [c[1] for c in centroids]
plt.scatter(cx, cy, marker='D', c='red', s=150, zorder=5, label='Centroids')

plt.xlabel("Top-half intensity")
plt.ylabel("Bottom-half intensity")
plt.title("K-means result (K=3) - our hand-coded algorithm")
plt.legend()
plt.tight_layout()
plt.show()

```



Q: Compare the K-means cluster assignments with the true digit labels shown earlier. Does the algorithm recover the three digit groups? Are there any misassignments?

YOUR ANSWER HERE

```
[12]: # Display the images in each cluster to see what K-means learned
# Build a dictionary: cluster_id -> list of image indices in that cluster
cluster_dict = {k: [] for k in range(3)}
for i, k in enumerate(assignments):
    cluster_dict[k].append(i)

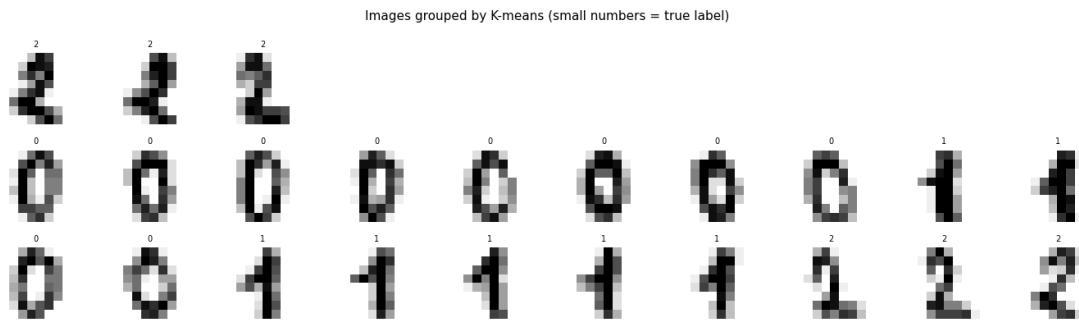
max_per_row = 10
fig, axes = plt.subplots(3, max_per_row, figsize=(14, 4))
for k in range(3):
    shown = cluster_dict[k][:max_per_row] # show at most 10 images per cluster
    for col, idx in enumerate(shown):
        axes[k, col].imshow(X_images[idx], cmap='gray_r')
        axes[k, col].axis('off')
        axes[k, col].set_title(f'{y_true[idx]}', fontsize=7)
```

```

# Hide any unused axes in this row
for col in range(len(shown), max_per_row):
    axes[k, col].axis('off')
    axes[k, 0].set_ylabel(f'Cluster {k}\n({len(cluster_dict[k])} pts)',
        ↪fontsize=9, rotation=0, labelpad=40)

plt.suptitle("Images grouped by K-means (small numbers = true label)",
    ↪fontsize=11)
plt.tight_layout()
plt.show()

```



Q: Look at the images inside each cluster. Do the clusters correspond to specific digits? Are there any images in the wrong cluster?

YOUR ANSWER HERE

Q: What happens if you run K-means with `seed=5` instead of `seed=42`? Try it and describe the result.

YOUR ANSWER HERE

1.1.4 Underfitting and Overfitting

K is a **hyperparameter** of K-means, just like the learning rate in gradient descent. Choosing K poorly leads to:

- **Underfitting** (K too small): the model merges distinct clusters instead of keeping them separate
- **Overfitting** (K too large): the model splits real clusters into meaningless sub-groups

```

[13]: # Collect 50 examples each of digits 0, 1, 2 (bigger dataset to demonstrate
    ↪underfitting and overfitting)
full_idx_0 = get_indices_for_digit(all_labels, 0, 50)
full_idx_1 = get_indices_for_digit(all_labels, 1, 50)
full_idx_2 = get_indices_for_digit(all_labels, 2, 50)

full_indices = full_idx_0 + full_idx_1 + full_idx_2

```

```

X_full_images = [all_images[i] for i in full_indices]
y_full         = [all_labels[i] for i in full_indices]

# Compute the same two features for the larger dataset
X_full = [[image_top_intensity(img), image_bottom_intensity(img)] for img in y_full]
X_full_images

print(f"Full subset size: {len(X_full)} points")

```

Full subset size: 150 points

```

[14]: # Run K-means with K = 1, 3, and 10, and compare
K_values = [1, 3, 10]

fig, axes = plt.subplots(1, 3, figsize=(15, 4))

for ax, K in zip(axes, K_values):
    asgn, cents, _ = run_kmeans(X_full, K=K, seed=0)

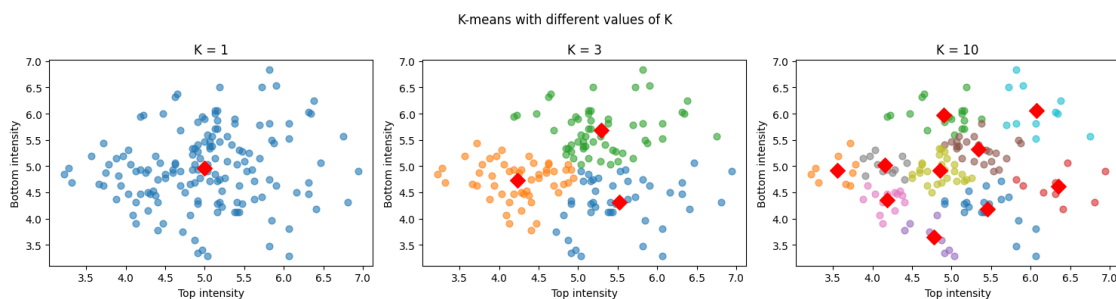
    for k in range(K):
        xs = [X_full[i][0] for i in range(len(X_full)) if asgn[i] == k]
        ys = [X_full[i][1] for i in range(len(X_full)) if asgn[i] == k]
        ax.scatter(xs, ys, alpha=0.6, s=40)

    cx = [c[0] for c in cents]
    cy = [c[1] for c in cents]
    ax.scatter(cx, cy, marker='D', c='red', s=100, zorder=5)
    ax.set_title(f'K = {K}')
    ax.set_xlabel("Top intensity")
    ax.set_ylabel("Bottom intensity")

plt.suptitle("K-means with different values of K", fontsize=12)
plt.tight_layout()
plt.show()

```

K-means converged at iteration 2
K-means converged at iteration 9
K-means converged at iteration 11



Q: Which value of K **underfits** the data? How can you tell?

YOUR ANSWER HERE

Q: Which value of K **overfits** the data? How can you tell?

YOUR ANSWER HERE

Q: Try different random seeds. What do you notice about the number of iterations until convergence for each K?

YOUR ANSWER HERE

1.1.5 The Elbow Method

One principled way to choose K is the **elbow method**: - Compute the inertia J for each value of K - Plot inertia vs. K - Look for the “elbow”: the point where the inertia stops decreasing rapidly

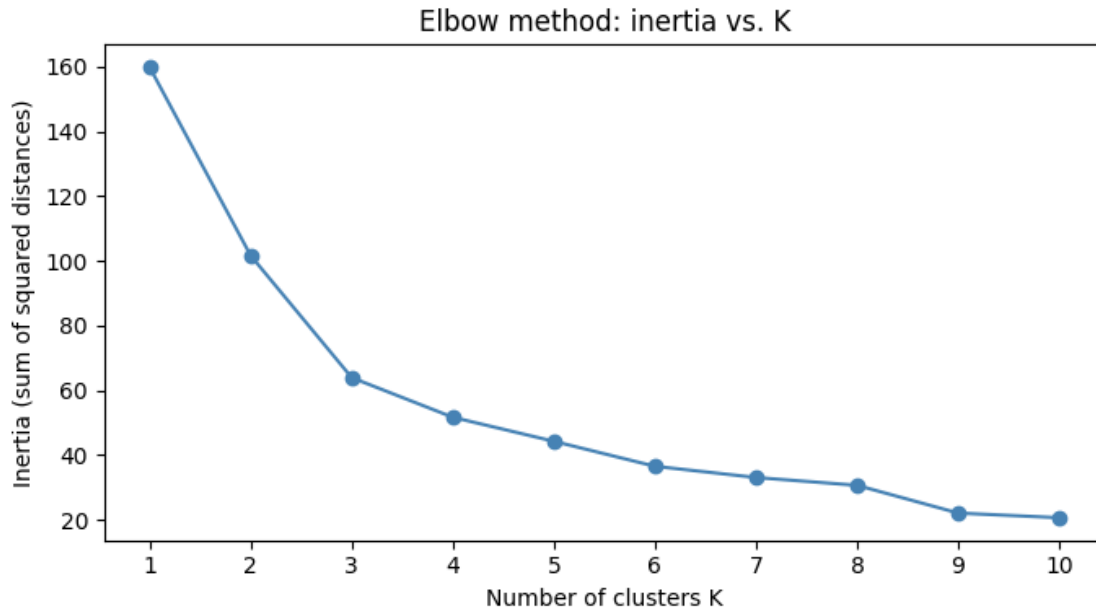
The intuition: adding more clusters always reduces inertia, but past the true number of clusters, the improvement becomes much smaller.

```
[15]: # Compute inertia for K = 1, 2, ..., 10
K_range = list(range(1, 11))
inertias = []

for K in K_range:
    asgn, cents, history = run_kmeans(X_full, K=K, seed=0)
    inertias.append(history[-1]) # final inertia after convergence

plt.figure(figsize=(7, 4))
plt.plot(K_range, inertias, marker='o', color='steelblue')
plt.xlabel("Number of clusters K")
plt.ylabel("Inertia (sum of squared distances)")
plt.title("Elbow method: inertia vs. K")
plt.xticks(K_range)
plt.tight_layout()
plt.show()
```

```
K-means converged at iteration 2
K-means converged at iteration 10
K-means converged at iteration 9
K-means converged at iteration 6
K-means converged at iteration 9
K-means converged at iteration 8
K-means converged at iteration 12
K-means converged at iteration 9
K-means converged at iteration 19
K-means converged at iteration 11
```

Q: Where is the “elbow” in the inertia curve? What value of K does it suggest?

YOUR ANSWER HERE

Q: Why does inertia always decrease as K increases? What happens to inertia when K equals the number of data points?

YOUR ANSWER HERE

1.1.6 Outlier Detection

K-means gives us more than just cluster labels. For each data point, we also know its **distance to the nearest centroid**. Points that are very far from any centroid are unusual: they don’t fit neatly into any cluster. These are **outliers**.

Outlier detection is a key application of unsupervised learning - Fraudulent credit card transactions (unusual spending pattern) - Faulty sensors (readings far outside typical range) - Corrupted or mislabeled images in a dataset

We’ll add a few **synthetic outlier points** to our dataset and see whether the distance score can find them.

```
[16]: # Add 4 synthetic outlier points (unusual for digit-like images)
outlier_points = [
    [0.5, 8.0], # very high bottom, near-zero top
    [7.5, 0.3], # very high top, near-zero bottom
    [5.0, 5.0], # equally high in both halves
    [0.2, 0.2], # almost completely blank
]
```

```

# Combine original data with outliers
X_with_outliers = X_full + outlier_points # list concatenation
n_normal = len(X_full)
n_outliers = len(outlier_points)

# True outlier flags: 0 = normal, 1 = outlier
is_outlier_true = [0] * n_normal + [1] * n_outliers

print(f"Dataset now has {len(X_with_outliers)} points ({n_normal} normal + {n_outliers} outliers)")

```

Dataset now has 154 points (150 normal + 4 outliers)

```

[17]: def outlier_scores(X, centroids, assignments):
    """
    The outlier score for each point is the squared Euclidean distance
    from the point to its assigned centroid. A high score means far from any
    cluster center so potentially an outlier.
    """
    scores = []

    for i in range(len(X)):
        # UNCOMMENT AND FILL IN:
        # k = assignments[i]
        # scores.append( ... ) # distance from X[i] to centroids[k]

        # REMOVE
        k = assignments[i]
        scores.append(squared_euclidean_distance(X[i], centroids[k]))

    return scores

# Run K-means on the combined dataset (with outliers) using K=3
asgn_out, cents_out, _ = run_kmeans(X_with_outliers, K=3, seed=0)

# Compute outlier scores
scores = outlier_scores(X_with_outliers, cents_out, asgn_out)

normal_scores = scores[:n_normal]
outlier_scores_vals = scores[n_normal:]

print("Normal points mean score: {:.2f}, max score: {:.2f}".format(
    sum(normal_scores) / len(normal_scores), max(normal_scores)))
print("Synthetic outlier scores:", [round(s, 2) for s in outlier_scores_vals])

```

K-means converged at iteration 9

Normal points mean score: 0.44, max score: 2.53

Synthetic outlier scores: [23.23, 18.71, 0.41, 34.65]

```
[18]: # Sorting the distance scores can give us the candidate outliers

def percentile(values, p):
    """
    Compute the p-th percentile of a list of numbers (0 <= p <= 100).
    Uses linear interpolation between adjacent sorted values.
    """
    sorted_vals = sorted(values)
    n = len(sorted_vals)
    idx = (p / 100.0) * (n - 1)
    lower = int(idx)
    upper = lower + 1
    if upper >= n:
        return sorted_vals[-1]
    frac = idx - lower
    return sorted_vals[lower] * (1 - frac) + sorted_vals[upper] * frac

# Flag as outlier: any point with score above the 95th percentile
threshold = percentile(scores, 95)
flagged = [score > threshold for score in scores]

print(f"Threshold (95th percentile): {threshold:.2f}")
print(f"Points flagged as outliers: {sum(flagged)}")
print(f"Of the {n_outliers} synthetic outliers, {sum(flagged[n_normal:])} were_
    ↪detected")
```

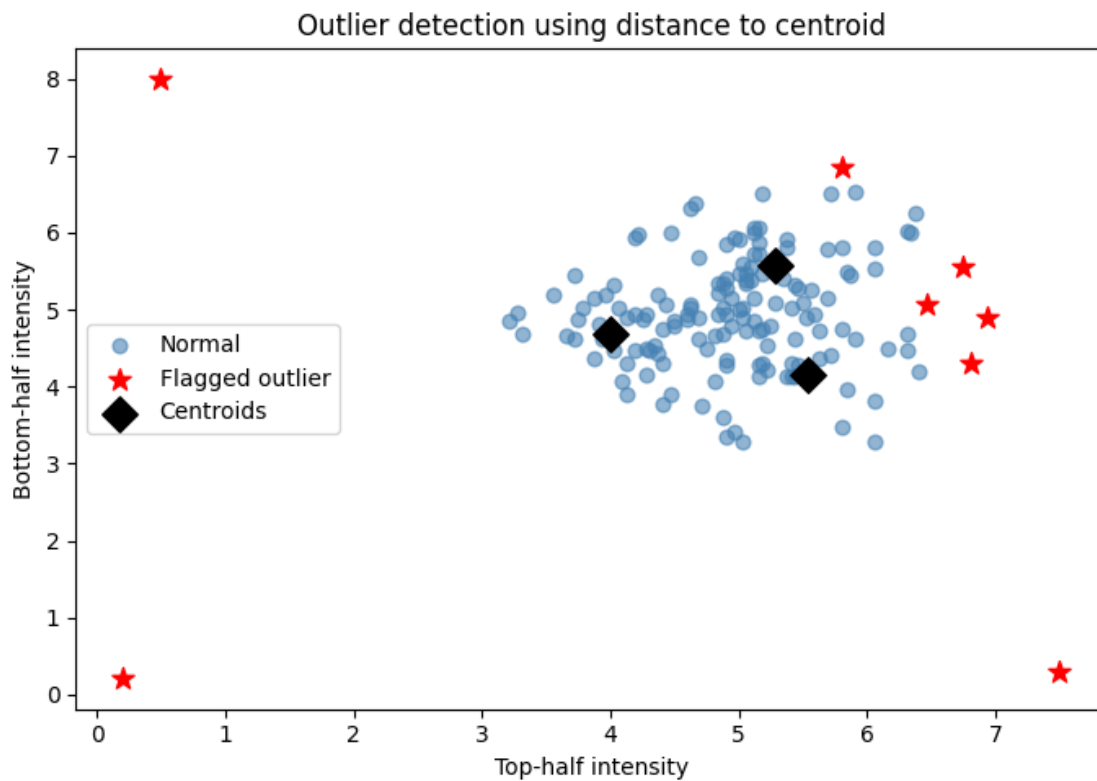
Threshold (95th percentile): 1.65
Points flagged as outliers: 8
Of the 4 synthetic outliers, 3 were detected

```
[19]: # Visualize: normal points in blue, flagged outliers as red stars
normal_xs = [X_with_outliers[i][0] for i in range(len(X_with_outliers)) if_
    ↪not flagged[i]]
normal_ys = [X_with_outliers[i][1] for i in range(len(X_with_outliers)) if_
    ↪not flagged[i]]
flagged_xs = [X_with_outliers[i][0] for i in range(len(X_with_outliers)) if_
    ↪flagged[i]]
flagged_ys = [X_with_outliers[i][1] for i in range(len(X_with_outliers)) if_
    ↪flagged[i]]
cent_xs = [c[0] for c in cents_out]
cent_ys = [c[1] for c in cents_out]

plt.figure(figsize=(7, 5))
plt.scatter(normal_xs, normal_ys, color='steelblue', alpha=0.6, s=40, _
    ↪label='Normal')
```

```
plt.scatter(flagged_xs, flagged_ys, color='red', s=100, zorder=5,
            label='Flagged outlier', marker='*')
plt.scatter(cent_xs, cent_ys, marker='D', c='black', s=120, zorder=6,
            label='Centroids')

plt.xlabel("Top-half intensity")
plt.ylabel("Bottom-half intensity")
plt.title("Outlier detection using distance to centroid")
plt.legend()
plt.tight_layout()
plt.show()
```



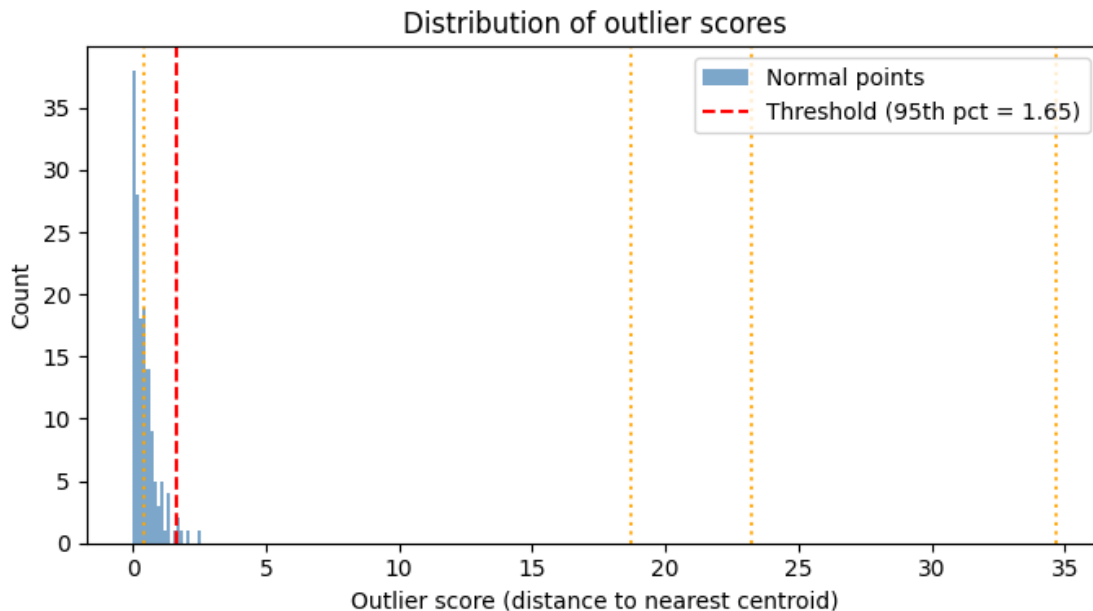
Q: Look at where the flagged outliers appear on the scatter plot. Why does their position make them unusual compared to the normal digit points?

YOUR ANSWER HERE

Q: What happens if you lower the threshold percentile to 80%? What happens if you raise it to 99%? What is the tradeoff?

YOUR ANSWER HERE

```
[20]: # Distribution of outlier scores for the normal points
plt.figure(figsize=(7, 4))
plt.hist(normal_scores, bins=20, color='steelblue', alpha=0.7, label='Normal_
points')
plt.axvline(threshold, color='red', linestyle='--',
            label=f'Threshold (95th pct = {threshold:.2f})')
for s in outlier_scores_vals:
    plt.axvline(s, color='orange', linestyle=':', alpha=0.9)
plt.xlabel("Outlier score (distance to nearest centroid)")
plt.ylabel("Count")
plt.title("Distribution of outlier scores")
plt.legend()
plt.tight_layout()
plt.show()
```



Q: Real-world outlier detection often faces a tradeoff called **precision vs. recall**. In the context of this demo: - What would a **false positive** mean (flagging a normal point as an outlier)? - What would a **false negative** mean (missing a true outlier)? - Which is more costly in a medical diagnosis setting? In a fraud detection setting?

YOUR ANSWER HERE

Q: In PA 1 you trained a **supervised** classifier on digit images with labels. Today you ran K-means, an **unsupervised** algorithm, on similar images without labels. Compare the two approaches: which gave better separation of the digit groups? What are the tradeoffs between using labels vs. not?

YOUR ANSWER HERE

1.1.7 Connecting to the Bigger Picture

We can describe the example we worked through today using the machine learning pipeline:

Pipeline step	K-means example
Problem formulation	“Find groups in unlabeled digit images”
Data collection	150 8×8 pixel images from <code>load_digits()</code>
Data processing	Compute top/bottom intensity features
Training	Iterative assign → update loop
Evaluation	Inertia; elbow method; compare to true labels
Deployment	Flag new unusual images as outliers

1.1.8 K-Means and the Five Big Ideas

- **Representation:** we chose which features to extract from the images (top/bottom intensity)
- **Learning:** the centroids are *learned* from data rather than designed by hand
- **Societal Impact:** outlier detection has real consequences (fraud, medical diagnosis, content moderation)